Galala University
Faculty of Computer Science and Engineering
CSE251 Software Engineering
Spring-2025/2026

Exam Management System

Software Engineering Project Report
UML and Diagrams Modeling & Full-Stack Implementation

Team:

Name	ID	Major
Samaa Mohamed Saleh	223101629	CE - Cybersecurity
Omar Khalifa	223101614	CE - Cybersecurity

Instructor: Dr. Nora Niazy
CSE251: Software Engineering Course Project

GALALA UNIVERSITY
Powered by
Arizona State University®

Table of Contents:

1. Introduction 3
1.1 Project Vision & Motivation 3
1.2 Project Scope 4
2. Software Requirements Specification (SRS) 4
2.1 System Overview 4
2.2 Functional Requirements 4
2.3 Non-Functional Requirements 6
3. System Architecture & Technology 7
3.1 Architectural Overview 7
3.2 Technology Stack 8
3.3 Middleware Pipeline 9
4. UML Diagrams 9
4.1 Context Diagram 9
Purpose 9
Description 10
4.2 Use Case Diagram 11
Purpose 11
Actors and Use Cases 12
Purpose 12
Swim Lanes & Flow 12
4.4 Sequence Diagram 14
Purpose 14
Lifelines Modeled 14
Purpose 17
Exam States 17
Session States 18
4.6 Data Flow Diagram (DFD) 18
Purpose 18
Major Processes (Level 1) 20
4.7 Entity Relationship Diagram (ERD) 20
Purpose 20
4.8 Class Diagram 21
Purpose 21

5.1 STRIDE Threat Model 23
5.2.1 Authentication 24
5.2.2 Authorization 24
5.2.3 Data Encryption 25
6. Database Design 25
6.1 Schema Overview 25
7. Implementation 26
7.1 Implementation Methodology 26
7.2 User Interface — Key Screens 26
8. Testing 27
8.1 Testing Strategy 27
8.2 Security Test Cases 28
8.3 Functional Test Results 29
9. Conclusion 30
9.1 Summary 30
9.3 Future Work 30

1. Introduction

1.1 Project Vision & Motivation

Academic institutions have long relied on paper based examination processes that are subjected to human error, difficult to scale if the number of students increases, and resistant to audit and change. ExamGuard was conceived as a comprehensive digital solution to this problem: a secure, role-aware, full-stack Exam Management System that digitizes and automates the entire examination lifecycle from question and exam creation and exam scheduling to proctored delivery, automatic grading, and results analytics.

1.2 Project Scope

ExamGuard is a web-based platform that serves three categories of users:

- **Super Administrator:** manages the system globally: user accounts, roles, groups, audit logs, and system configuration.
- **Instructor (Teacher):** authors and publishes exams, monitors sessions, manually grades open-ended responses, and communicates with students through encrypted messaging.
- **Student:** registers for exams, takes them within enforced time windows, views grades and performance analytics, and communicates with instructors.

2. Software Requirements Specification (SRS)

2.1 System Overview

ExamGuard is built as a modular website running on Node.js with Express.js, backed by a SQLite3 relational database (upgradeable to PostgreSQL), and served over HTTPS with WebSocket support for real-time features. The system enforces a strict separation of concerns across four layers: Frontend, Application (API), Middleware, and Data.

2.2 Functional Requirements

The following table defines all functional requirements gathered during requirements analysis:

ID	Requirement	Actor
FR-01	Users can register with a validated username (3–30 chars, no spaces), valid email address, and a strong password (min 8 chars, 1 uppercase, 1 digit)	All Users
FR-02	Users can log in using username and password with CAPTCHA verification and optional two-factor authentication (TOTP/device code)	All Users
FR-03	Instructors can create exams specifying title, course, description, duration, time window, and configuration options	Instructor
FR-04	Instructors can add MCQ, True/False, and Open-Ended questions to exams, with configurable marks and ordering	Instructor
FR-05	Instructors can publish, edit, and delete their own exams, published exams become immutable snapshots	Instructor
FR-06	Students can view all exams they are enrolled in and take any published exam within its configured access window	Student
FR-07	The system automatically grades MCQ and True/False answers upon submission and calculates total score and percentage	System
FR-08	The system enforces exam time limits server-side and auto-submits the session when the timer expires	System
FR-09	Students can view their complete exam history, scores, and answer review for completed sessions	Student
FR-10	The Super Admin can create, edit, lock, and delete user accounts; assign roles; and manage student groups	Admin

FR-11	All users can participate in real-time encrypted chat; messages are stored with AES-256 encryption at rest	All Users
FR-12	Role-specific dashboards display performance analytics: exam statistics, score distributions, and pass rates	All Users
FR-13	Instructors can manually grade Open-Ended questions and override system-calculated scores with comments	Instructor
FR-14	The system maintains an immutable append-only audit log recording all security-sensitive events	System
FR-15	Anti-cheat mechanisms detect and record tab-switching, focus loss, and screen-share violations	System

2.3 Non-Functional Requirements

Category	Requirement
Security	AES-256-CBC encryption for passwords and messages, RSA-2048 for key exchange, PBKDF2-SHA512 for password hashing, CSRF tokens on all state-changing requests, rate limiting (100 req/min), RBAC and ABAC middleware, HttpOnly session cookies; parameterized SQL queries to prevent injection.
Performance	Page load time under 2 seconds on standard hardware, support for 100+ concurrent active exam sessions, server-side timer with WebSocket tick delivery for real-time accuracy.
Usability	Responsive UI compatible with desktop and mobile browsers, role-specific dashboards with context-appropriate navigation, consistent design language with accessibility considerations.

Reliability	Auto-save exam answers every 30 seconds via WebSocket, session recovery after network interruption using session tokens, SQLite database to prevent write conflicts.
Scalability	Modular route and controller structure, SQLite schema designed to be portable to PostgreSQL with minimal changes, Redis-ready session store for horizontal scaling.
Auditability	All security-sensitive events (logins, exam submissions, grade changes, admin actions) recorded in an immutable append-only audit log with IP address, user agent, severity, and timestamp.
Maintainability	Separate files per functional concern (routes, controllers, middleware, models); documented API endpoints; clear error handling and logging throughout.

3. System Architecture & Technology

3.1 Architectural Overview

ExamGuard follows a layered Model-View-Controller (MVC) architecture with a dedicated security middleware layer. The four layers are:

- Frontend Layer: HTML5, CSS3, and vanilla JavaScript with EJS server-side templating. Responsible for rendering role-specific views and collecting user input.
- Application Layer: Node.js with Express.js. Handles HTTP routing, business logic, WebSocket management, and API responses.
- Middleware Layer: A chain of security middleware applied to every request: JWT authentication, RBAC/ABAC authorization, CSRF validation, rate limiting, input sanitization, and audit logging.

- Data Layer: SQLite3 via the better-sqlite3 driver. Twelve relational tables with parameterized queries, foreign key constraints, and referential integrity.

3.2 Technology Stack

Component	Technology	Purpose / Rationale
Backend Server	Node.js + Express.js	Non-blocking I/O, extensive middleware ecosystem, JSON-native API handling
Database	SQLite3 (better-sqlite3)	Zero-configuration relational DB with WAL mode; portable to PostgreSQL
Encryption	Node.js crypto module	AES-256-CBC for data at rest; RSA-2048 for key exchange; PBKDF2-SHA512 for passwords
Authentication	jsonwebtoken (JWT)	Stateless auth with 15-min access tokens and 3-day refresh tokens in HttpOnly cookies
Templating	EJS (Embedded JS)	Server-side rendering for SEO-friendly, CSP-compatible HTML output
Input Validation	validator.js + html.escape()	Schema validation and XSS prevention on all user-supplied input
Rate Limiting	express-rate-limit	Brute-force and DoS protection, 100 requests/min per IP
Real-time Comms	WebSocket (ws)	Live timer ticks, answer auto-save, anti-cheat event streaming

Email	SendGrid API	Transactional emails: results, regrade notices, account confirmations
2FA	TOTP (RFC 6238)	Time-based one-time passwords for multi-factor authentication

3.3 Middleware Pipeline

Every HTTP request passes through the following middleware chain before reaching a route controller:

Pipeline: Rate Limiter > CSRF Validator > XSS Sanitizer > JWT Auth > RBAC/ABAC > Route Controller > Audit Logger

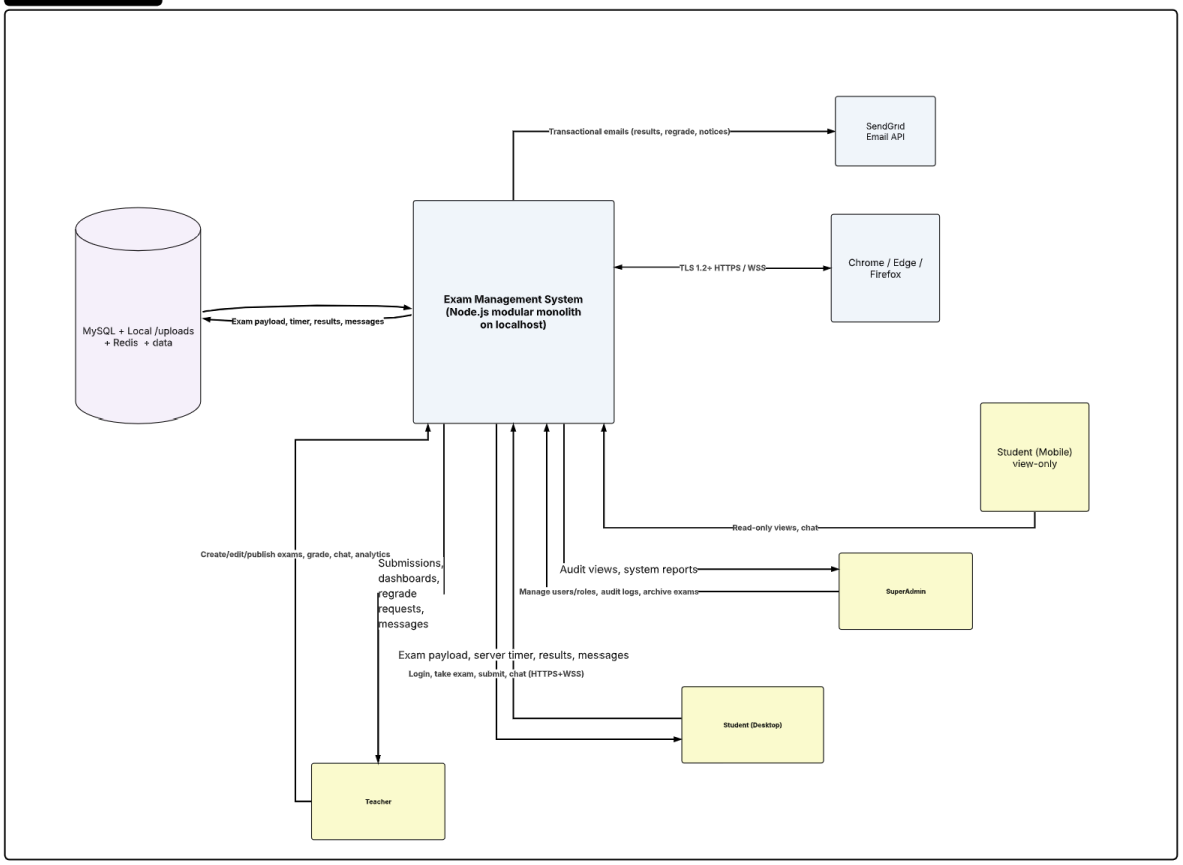
4. UML Diagrams

4.1 Context Diagram

Purpose

The Context Diagram (also known as a Level-0 DFD or System Context Diagram) defines the system boundary of ExamGuard and documents all external entities that interact with it, along with the nature of those interactions. It provides the highest-level view of the system without exposing any internal processes.

1. Context Diagram



Description

The diagram identifies the following external entities and their information flows:

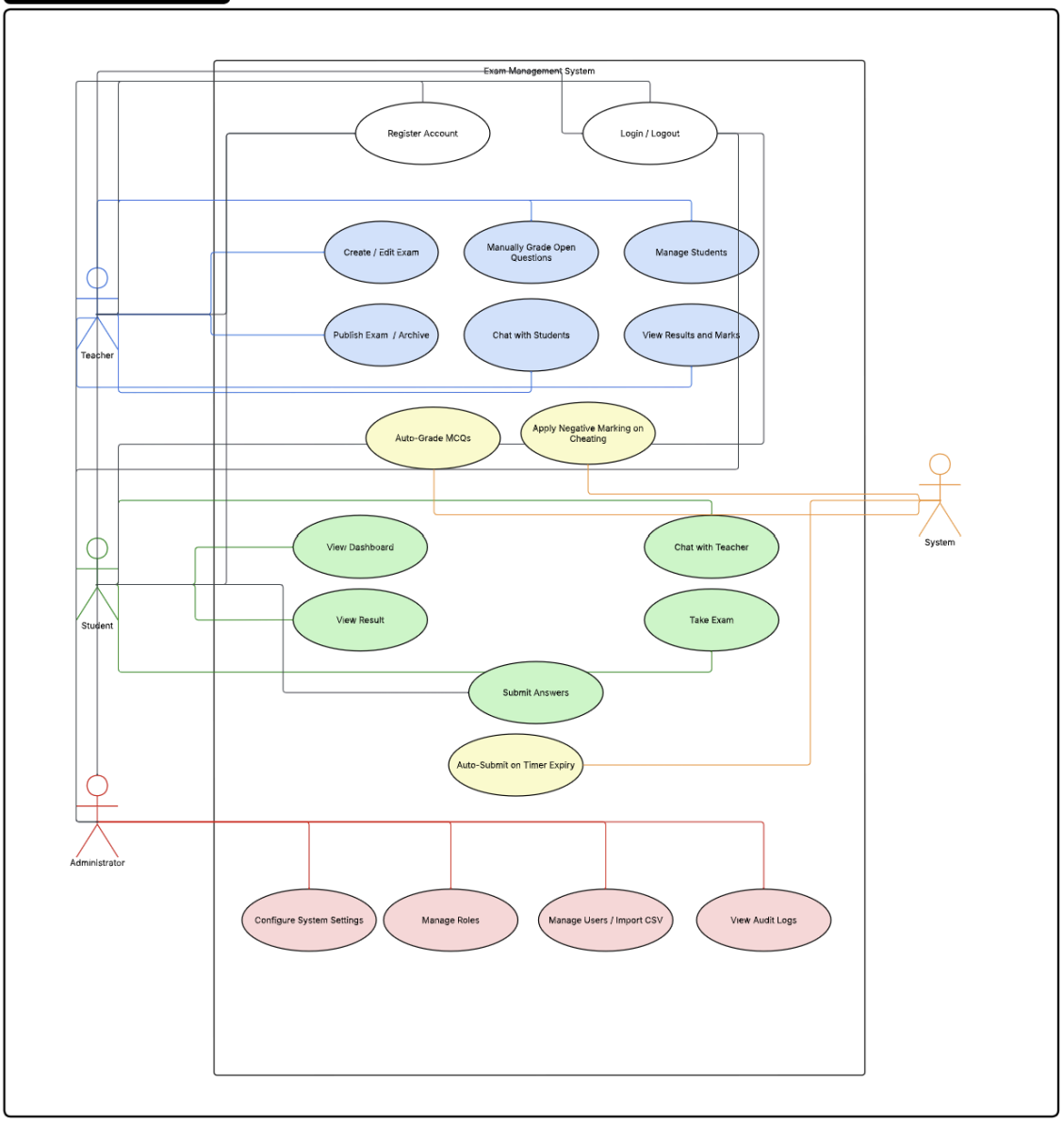
- Student (Desktop / Mobile)
- Teacher (Instructor)
- Super Admin
- SendGrid Email API — an external SMTP relay used by the system to send transactional emails (result notifications, regrade notices, account confirmations) to users.

4.2 Use Case Diagram

Purpose

The Use Case Diagram shows the system's functional behavior from the POV of its actors. It defines what each actor can do and maps out all system use cases along with their relationships.

2. Use Case Diagram



Actors and Use Cases

- Student: Register Account, Login/Logout, Take Exam, Submit Answers, View Results, Chat with Teacher.
- Teacher: Login/Logout, Create/Edit Exam, Publish/Archive Exam, Manually Grade Open Questions, View Analytics, Chat with Students, Manage Student Groups.
- Administrator: Login/Logout, Manage Users, Import Users via CSV, Manage Roles, Configure System Settings, View Audit Logs, View Results.
- System (automated): Auto-Grade MCQs, Auto-Submit on Timer Expiry, Apply Anti-Cheat Penalties, Send Notifications.

4.3 Activity Diagram

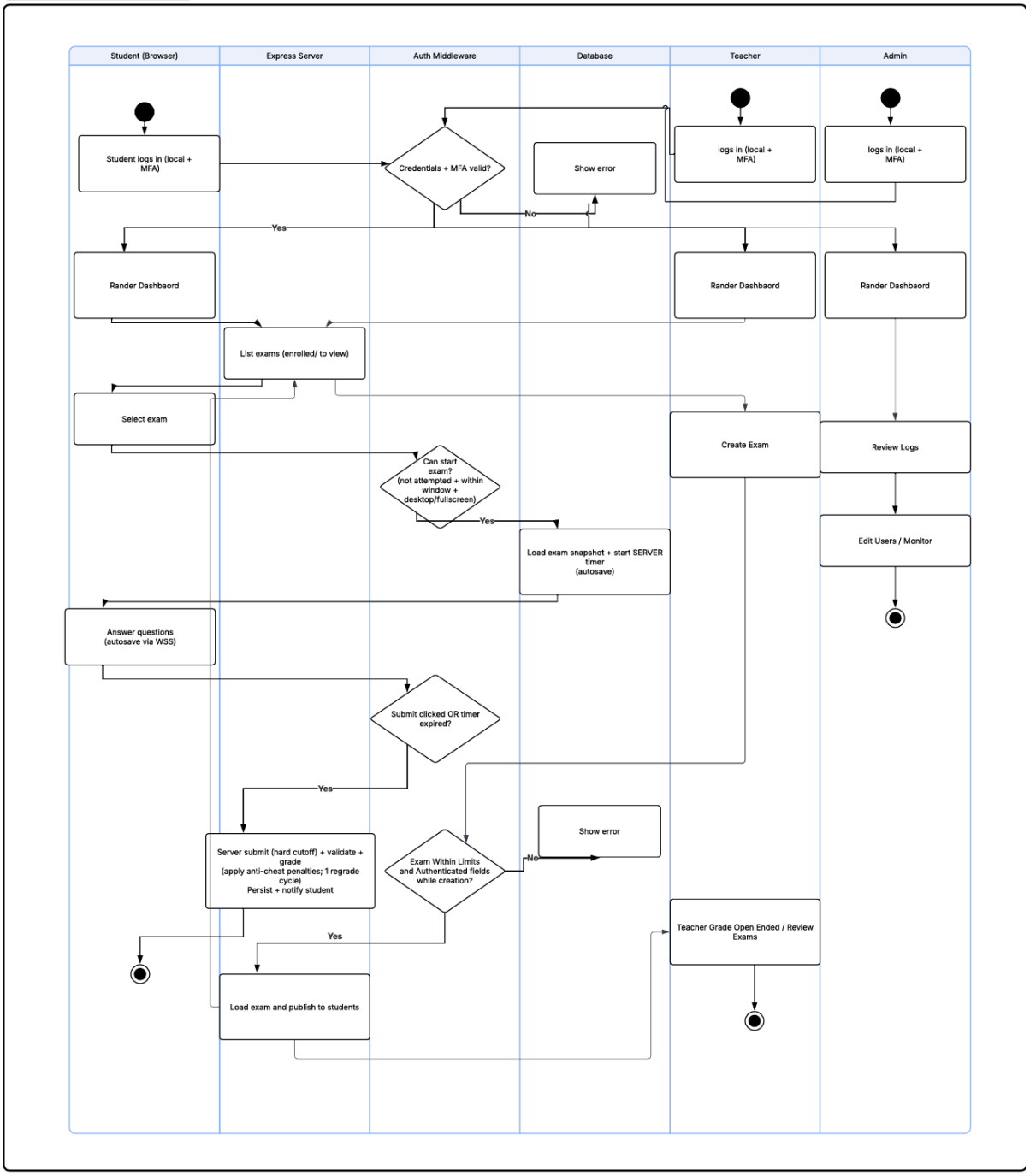
Purpose

The Activity Diagram models the dynamic flow of activities (actions and decisions) within the system for the primary end-to-end scenario: a student taking an exam. It uses swim lanes to separate responsibilities across system components and actors.

Swim Lanes & Flow

- Student (Browser): Initiates login, selects exam, answers questions, and clicks submit.
- Express Server: Validates credentials, loads exam snapshot, enforces time limits, processes submission, and triggers grading.
- Auth Middleware: Validates JWT tokens, checks MFA if enabled, and enforces session state.
- Database: Persists draft answers, exam sessions, grades, and audit log entries.
- Teacher: Creates exam, reviews and grades open-ended responses.
- Admin: Monitors the system, manages users, and reviews logs.

3. Activity Diagram



4.4 Sequence Diagram

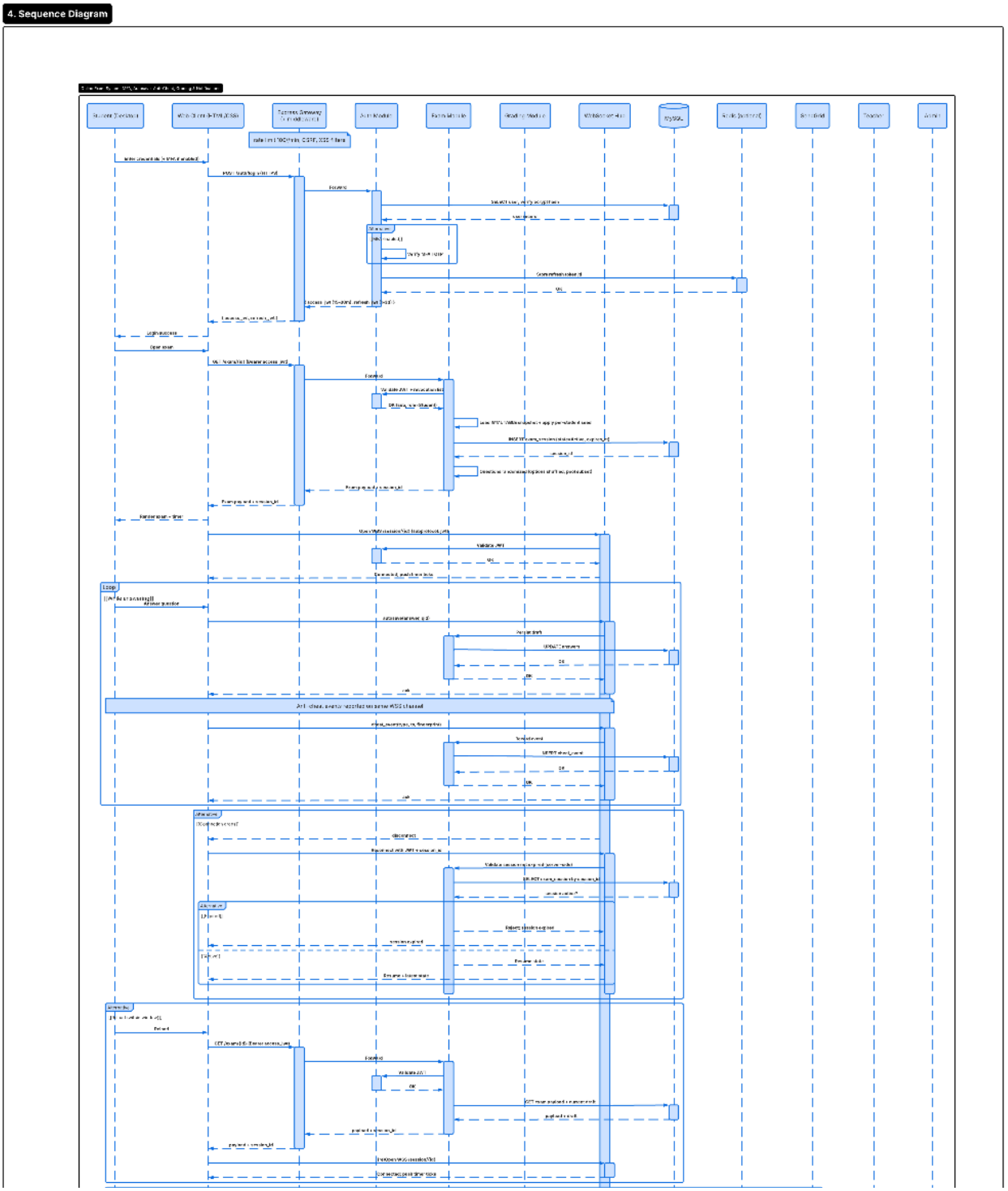
Purpose

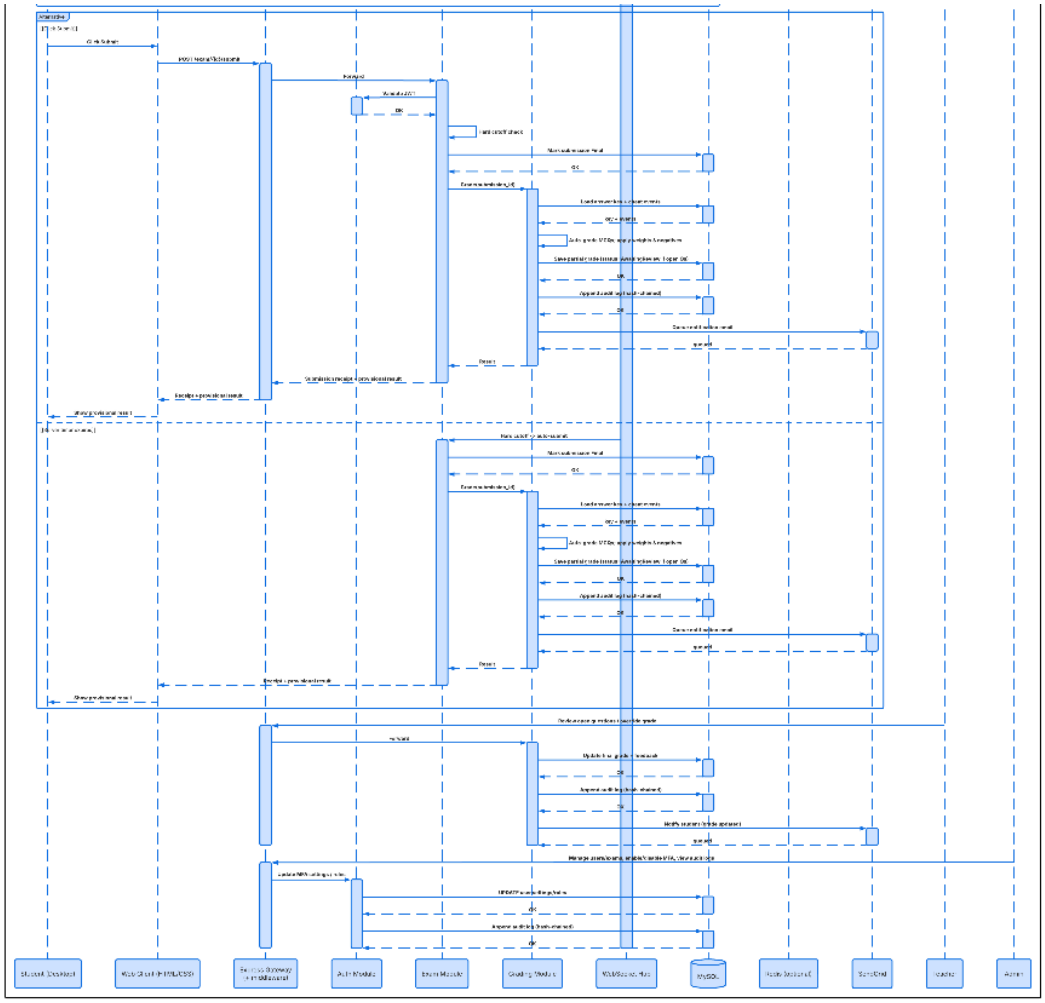
The Sequence Diagram is the most detailed behavioral diagram. It models the chronological sequence of messages exchanged between system components during a complete exam session.

Lifelines Modeled

The diagram includes eight lifelines: Student (Desktop Browser), Web Client (HTML/CSS), Express Gateway + Middleware, Auth Module, Exam Module, Grading Module, WebSocket Hub, MySQL/SQLite Database, Redis (optional), and SendGrid Email Service.

4. Sequence Diagram

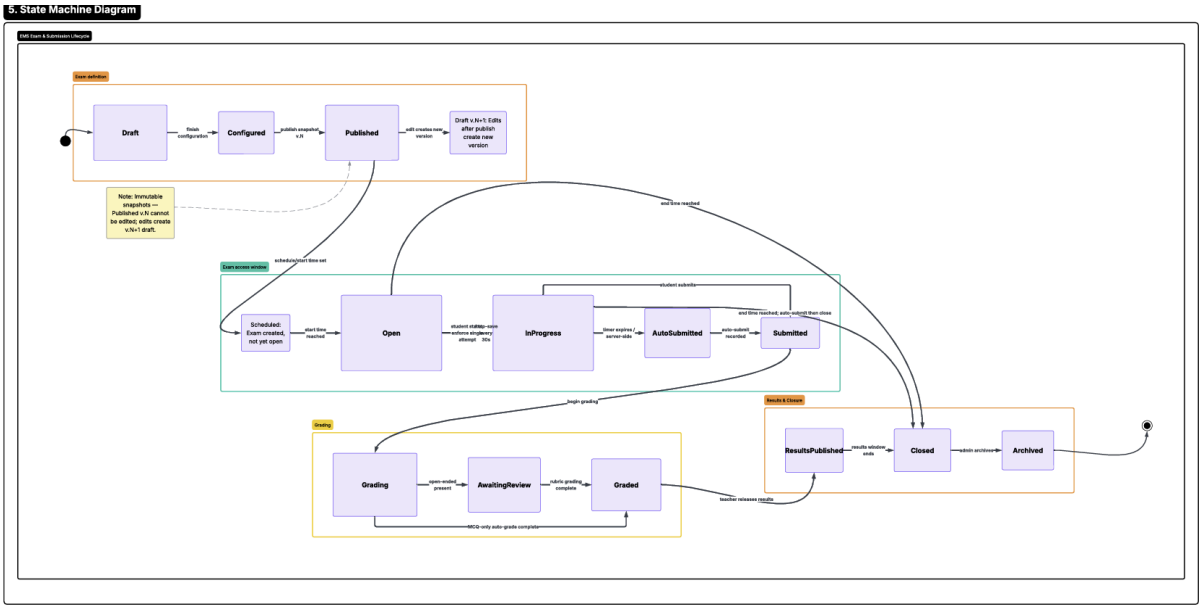




4.5 State Machine Diagram

Purpose

The State Machine Diagram models the lifecycle states of two core entities: the Exam (from authoring to archival) and the Exam Session (from the student starting to grade publication). It defines every state, every transition trigger, and every guard condition.



Exam States

- Draft: Exam is being authored; all fields are editable.
- Configured: Configuration is finalized but not yet published.
- Published (v.N): An immutable snapshot is created; edits after this point create a new version (Draft v.N+1). Multiple version coexistence is supported.
- Scheduled: The exam exists but the access window has not yet opened.
- Open: The access window is active; students can start the exam.

Session States

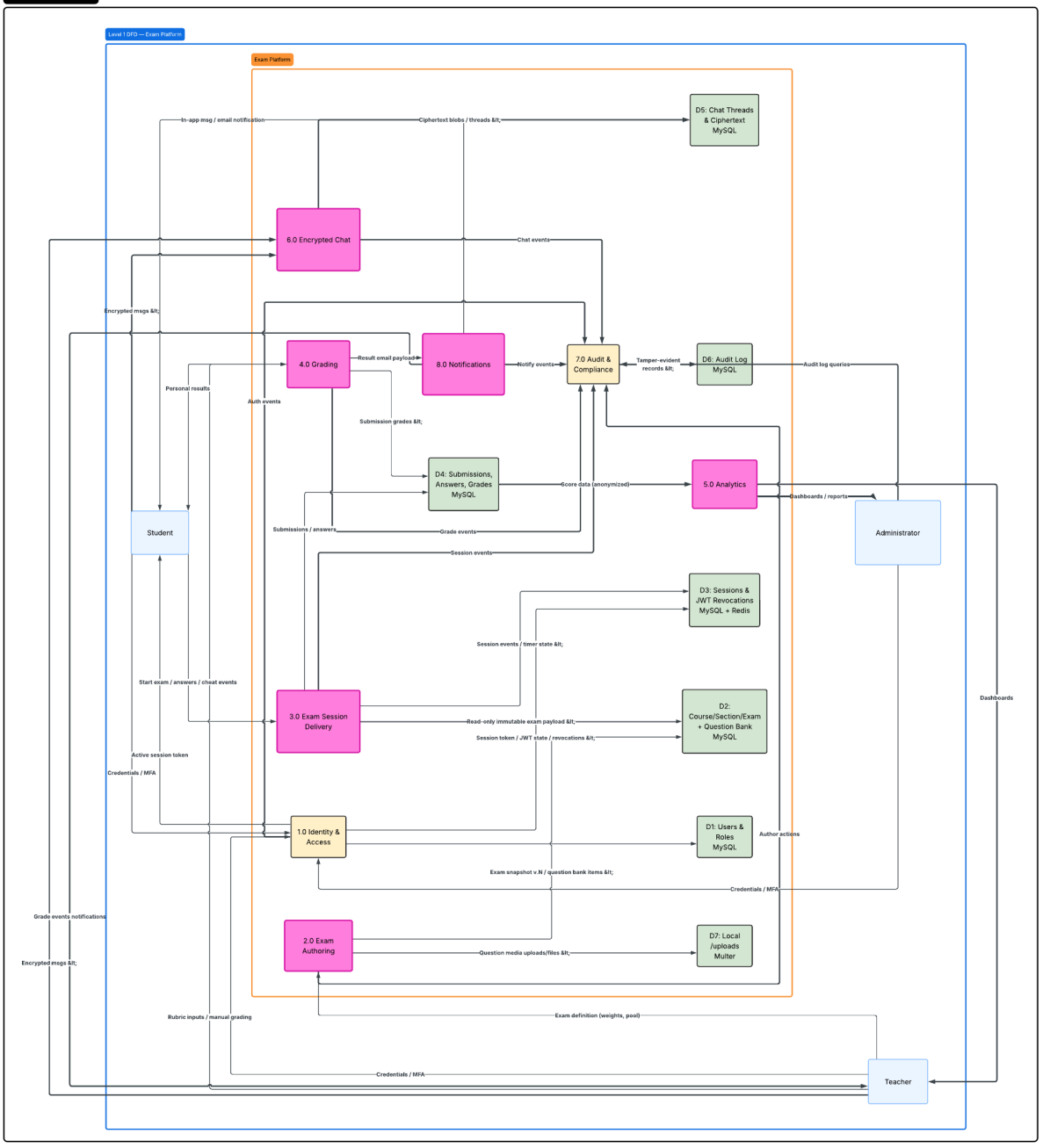
- In Progress: Student has started; auto-save is active every 30 seconds.
- Submitted: Student clicked Submit.
- Auto-Submitted — Server timer expired; server-side auto-submission executed.
- Grading: MCQ auto-grading is running.
- Awaiting Review: Open-ended questions are present; manual grading required.
- Graded: All questions have been scored.
- Results Published: Teacher released results to students.
- Closed / Archived: Exam window ended; exam archived for records.

4.6 Data Flow Diagram (DFD)

Purpose

The Data Flow Diagram (DFD) models how data moves through the system. It does not show control flow or sequencing. The DFD for ExamGuard is drawn at Level 1 (major processes and data stores).

6. DFD Level 1


Level 1 DFD — Exam Platform
Exam Platform
D5: Chat Threads & Ciphertext MySQL
In-app msg / email notification
Ciphertext blobs / threads <
6.0 Encrypted Chat
Chat events
Encrypted msgs <
4.0 Grading
Result email payload
8.0 Notifications
Notify events
7.0 Audit & Compliance
Tamper-evident records <
D6: Audit Log MySQL
Audit log queries
Personal results
Auth events
Submission grades <
D4: Submissions, Answers, Grades MySQL
Score data (anonymized)
5.0 Analytics
Dashboards / reports
Student
Submissions / answers
Grade events
Administrator
Session events
D3: Sessions & JWT Revocations MySQL + Redis
Session events / timer state <
Start exam / answers / cheat events
3.0 Exam Session Delivery
Read-only immutable exam payload <
Session token / JWT state / revocations <
D2: Course/Section/Exam + Question Bank MySQL
Dashboards
Active session token
Credentials / MFA
1.0 Identity & Access
D1: Users & Roles MySQL
Author actions
Exam snapshot v.N / question bank items <
Credentials / MFA
Grade events notifications
2.0 Exam Authoring
Question media uploads/files <
D7: Local /uploads Multer
Encrypted msgs <
Rubric inputs / manual grading
Exam definition (weights, pool)
Credentials / MFA
Teacher

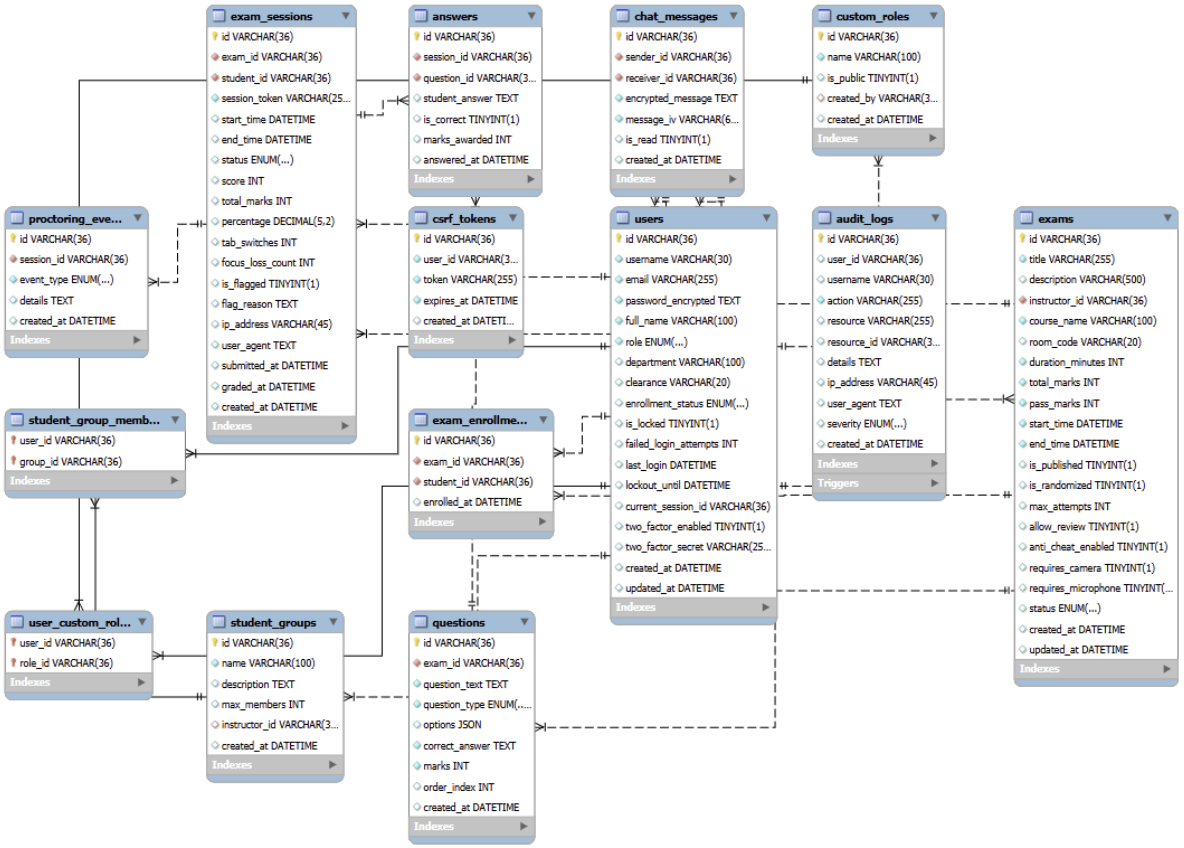
Major Processes (Level 1)

- 1.0 Identity & Access: Handles authentication and authorization. Reads/writes from D1: Users & Roles.
- 2.0 Exam Authoring: Manages exam creation, question bank, and publishing. Reads/writes from D2: Course/Section/Exam and D7: Local /uploads.
- 3.0 Exam Session Delivery: Serves exam snapshots, manages WebSocket session state, handles answer auto-save. Reads from D2, writes to D3: Sessions.
- 4.0 Grading: Auto-grades MCQ/TF answers, applies cheat penalties, supports manual instructor grading. Reads from D4: Submissions, Answers, Grades.
- 5.0 Analytics: Aggregates anonymized score data for dashboard display. Reads from D4.
- 6.0 Encrypted Chat: Handles message encryption/decryption and delivery. Reads/writes D5: Chat Threads & Ciphertext.
- 7.0 Audit & Compliance: Appends evident records for all security events. Writes to D6: Audit Log (append-only).
- 8.0 Notifications: Queues and sends transactional emails via SendGrid. Triggered by Grading and Admin actions.

4.7 Entity Relationship Diagram (ERD)

Purpose

The Entity Relationship Diagram (ERD) models the persistent data structure of ExamGuard. It defines all twelve database entities (tables), their attributes, primary and foreign keys, and the cardinality of relationships between them.

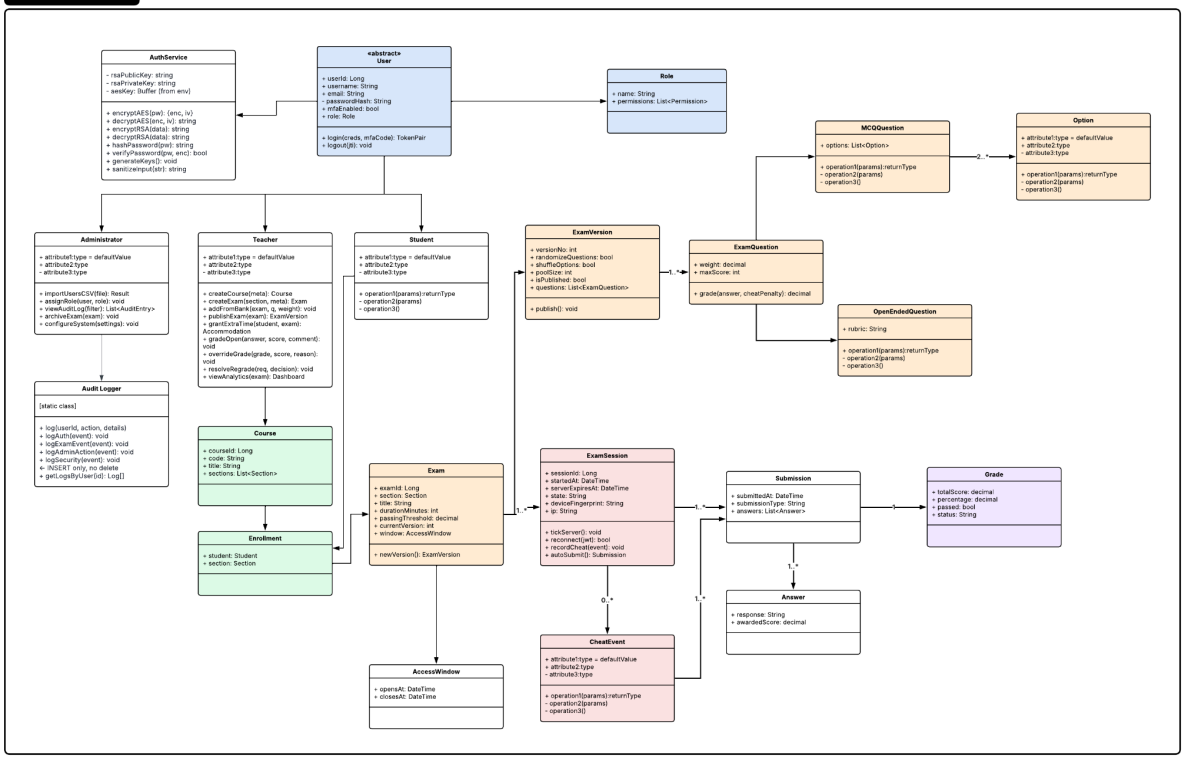


4.8 Class Diagram

Purpose

The Class Diagram models the object-oriented design of ExamGuard. It defines the system's classes, their attributes, methods, access modifiers, and the relationships between them.

8. Class Diagram



5. Security Design

5.1 STRIDE Threat Model

The STRIDE framework was applied to the system's Data Flow Diagram to identify potential threats across six categories.

STRIDE	Threat	Asset at Risk	Mitigation
Spoofing	An attacker impersonates a legitimate user by stealing or guessing credentials	User session tokens, JWT access tokens	bcrypt/PBKDF2 password hashing; short-lived JWTs (15 min); refresh token rotation; account lockout after 5 failed attempts; CAPTCHA on login; 2FA option
Tampering	An attacker modifies exam answers after submission to improve their score	exam_sessions, answers tables	Server-side grading only — client never submits scores; all answers validated server-side; database triggers; audit log on all grade changes
Repudiation	A user denies performing an action (e.g., submitting an exam or changing a grade)	Audit log, exam_sessions	Immutable append-only audit log with hash-chaining; all actions tagged with user ID, IP, user agent, and timestamp; database trigger prevents DELETE on audit_logs
Information Disclosure	An attacker reads encrypted passwords, exam answers, or chat messages from the database	Users table (password_encrypted), chat_messages (encrypted_message)	AES-256-CBC encryption with random IV per record; RSA-2048 key exchange; passwords never stored in plaintext; HttpOnly cookies prevent JS access to tokens

Denial of Service	An attacker floods the server with login requests to exhaust resources or lock out users	Authentication endpoints, exam session endpoints	express-rate-limit: 100 req/min per IP; WebSocket connection throttling; server-side session management; CAPTCHA on auth routes
Elevation of Privilege	A student accesses instructor or admin routes by manipulating their JWT role claim	RBAC middleware, JWT role claim	Role stored in database, not only in JWT; RBAC middleware verifies DB role on every request; ABAC adds department+clearance checks for sensitive routes; admin registration disabled via form

5.2 Implemented Security Controls

5.2.1 Authentication

- Password hashing with PBKDF2-SHA512 (100,000 iterations) — resistant to brute-force and rainbow table attacks.
- JWT access tokens (15-min TTL) with refresh tokens (3-day TTL) stored in HttpOnly, SameSite=Strict cookies.
- Token revocation list checked on every authenticated request.
- Account lockout: 5 failed attempts triggers a 15-minute lockout (configurable).
- Two-Factor Authentication (TOTP RFC 6238) available for all users.

5.2.2 Authorization

- Role-Based Access Control (RBAC) — three roles (super_admin, instructor, student) enforced via Express middleware on every route.
- Attribute-Based Access Control (ABAC) — finance-department and clearance-level checks for sensitive analytics routes (time-restricted: 09:00–18:00).
- Custom roles and student groups allow fine-grained permission grants without elevating the base role.

5.2.3 Data Encryption
5.2.4 Input Validation & XSS Prevention
5.2.5 CSRF Protection
5.2.6 Audit Logging

6. Database Design

6.1 Schema Overview

The ExamGuard database consists of twelve tables organized around five functional domains: Identity & Access, Exam Management, Session & Grading, Communication, and Compliance & Audit.

Table	Domain	Key Columns
users	Identity & Access	id, username, email, password_encrypted, role, department, clearance, is_locked, two_factor_secret
exams	Exam Management	id, title, instructor_id (FK), course_name, duration_minutes, start_time, end_time, is_published, status
questions	Exam Management	id, exam_id (FK), question_text, question_type, options (JSON), correct_answer, marks
exam_enrollments	Exam Management	id, exam_id (FK), student_id (FK), enrolled_at
exam_sessions	Session & Grading	id, exam_id (FK), student_id (FK), status, score, percentage, tab_switches, is_flagged, graded_at
answers	Session & Grading	id, session_id (FK), question_id (FK), student_answer, is_correct, marks_awarded

proctoring_events	Session & Grading	id, session_id (FK), event_type, details, created_at
chat_messages	Communication	id, sender_id (FK), receiver_id (FK), encrypted_message, message_iv, is_read
audit_logs	Compliance & Audit	id, user_id, username, action, resource, ip_address, severity, created_at
csrf_tokens	Identity & Access	id, user_id (FK), token, expires_at
custom_roles	Identity & Access	id, name, is_public, created_by
student_groups	Exam Management	id, name, description, max_members, instructor_id (FK)

7. Implementation

7.1 Implementation Methodology

The project followed an incremental development methodology, building and validating the system in layers:

database schema > authentication > RBAC/ABAC > core exam features > grading > analytics > chat.

7.2 User Interface — Key Screens

The following screens were implemented as part of the front-end prototype:

- Login Page: Username/password fields, CAPTCHA verification widget, optional 2FA device code input, CSRF hidden field, and branded ExamGuard Pro header.
- Registration Page: Validated registration form with real-time password strength indicator and role selection (student/instructor only; admin cannot be self-selected).

- Student Dashboard: Lists enrolled exams with status indicators (upcoming, open, completed), score summaries, and a link to the encrypted chat.
- Instructor Dashboard: Shows created exams with session statistics, a question bank manager, and a grading queue for open-ended responses.
- Exam Taking View: Full-screen exam interface with a countdown timer, question navigator, anti-cheat warning banners, and auto-save indicator.
- Admin Panel: Tabbed interface with user management table, audit log viewer with severity filtering, and group/role management.
- Encrypted Chat: Real-time messaging interface with instructor-student pairing; all messages encrypted before transmission and storage.

8. Testing

8.1 Testing Strategy

We followed a layered testing strategy because of the system's layered architecture. Tests were designed to verify both functional correctness and security properties at each layer.

- Unit Testing: Individual functions tested in isolation: validation functions, encryption/decryption round-trips, grading logic, and RBAC decision functions.
- Integration Testing: Routes tested end-to-end using HTTP requests against a test database instance, verifying middleware chain correctness.
- Security Testing: Manual penetration testing of key attack surfaces: SQL injection via input fields, XSS via question text and usernames, CSRF bypass attempts, privilege escalation via URL manipulation, and brute-force login.
- User Acceptance Testing: Manual walkthrough of all use cases by each role to verify functional completeness.

8.2 Security Test Cases

Test Case	Attack Vector / Input	Expected & Actual Result
SQL Injection on Login	Username: ' OR '1'='1	Query uses parameterized statements → injection neutralized → login rejected ✓
XSS via Exam Question	<script>alert(1)</script> in question text	html.escape() applied on input → script tags rendered as literal text ✓
CSRF bypass on exam submission	POST /submit without _csrf token	CSRF middleware rejects request with 403 Forbidden ✓
Privilege escalation (URL manipulation)	Student GETs /admin/users directly	RBAC middleware rejects with 403; audit log entry created ✓
Brute-force login	100+ failed login attempts	Rate limiter blocks after 100 req/min; account locked after 5 bad passwords ✓
Session token replay	Reuse expired JWT access token	Token revocation list check rejects expired token; 401 Unauthorized ✓
Admin self-registration	POST /register with role=super_admin	Server strips and ignores role parameter; user created as 'student' ✓
Answer tampering	Client-side modification of score before POST	Server recalculates score from stored answers; client-supplied score ignored ✓

8.3 Functional Test Results

FR ID	Test Scenario	Status
FR-01	Register with invalid email, short password, username with spaces	**PASS ✓**
FR-02	Login with correct credentials + CAPTCHA + 2FA code	**PASS ✓**
FR-03	Create exam with title, duration, time window	**PASS ✓**
FR-04	Add MCQ, TF, and Open-Ended questions	**PASS ✓**
FR-05	Publish exam; verify immutability	**PASS ✓**
FR-06	Student takes published exam within window	**PASS ✓**
FR-07	Submit MCQ answers; verify auto-grading	**PASS ✓**
FR-08	Let timer expire; verify auto-submission	**PASS ✓**
FR-09	View grade history and answer review	**PASS ✓**
FR-10	Admin locks a user account	**PASS ✓**
FR-11	Send encrypted message; verify ciphertext in DB	**PASS ✓**
FR-12	View analytics dashboard per role	**PASS ✓**

9. Conclusion

9.1 Summary

- Complete requirements engineering: 15 functional requirements and 7 non-functional requirement categories fully documented, traced, and implemented.
- Eight comprehensive UML diagrams providing complete structural, behavioral, and data-centric coverage of the system.
- A fully functional full-stack implementation: Node.js backend, SQLite database with 12 tables, RESTful API, EJS-rendered frontend, and WebSocket real-time communication.
- Security-first design with defense-in-depth: AES-256-CBC encryption, RSA-2048 key exchange, PBKDF2 hashing, JWT + HttpOnly cookies, RBAC, ABAC, CSRF protection, rate limiting, XSS prevention, SQL injection prevention via parameterized queries, and an immutable audit trail.
- STRIDE threat model applied with documented mitigations for all six threat categories.
- All 12 functional requirements verified through testing; all 8 security test cases passed.

9.3 Future Work

- Migrate from SQLite to PostgreSQL for production-scale deployments.
- Integrate Redis for session caching and WebSocket pub/sub at scale.
- Add AI-powered plagiarism detection for open-ended answers.
- Implement video proctoring via WebRTC for high-stakes examinations.
- Add OAuth 2.0 / OIDC support for institutional Single Sign-On (SSO).
- Introduce a question bank with difficulty tagging and adaptive exam generation.

References

1. Pressman, R. S. & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
2. OWASP Foundation. (2021). OWASP Top Ten.
3. Microsoft. (2009). The STRIDE Threat Model. Microsoft Threat Modeling Documentation.
4. NIST. (2020). NIST Special Publication 800-63B: Digital Identity Guidelines.
5. Express.js Documentation.
6. Node.js Crypto Module Documentation.
7. RFC 6238 — TOTP: Time-Based One-Time Password Algorithm. IETF.
8. Lucidchart. (2025). UML Diagramming Tool.